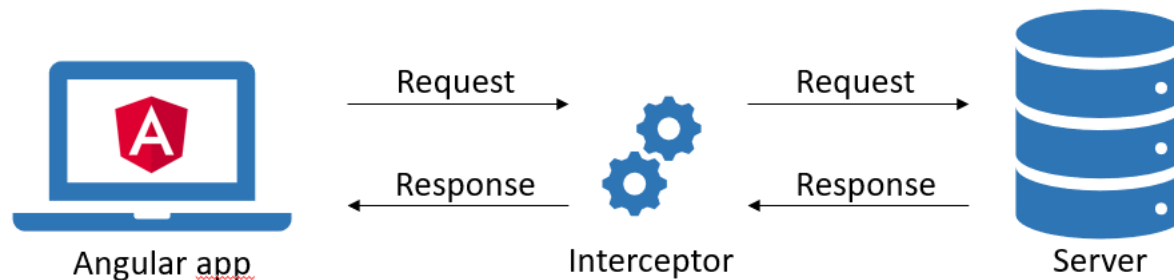


What is an Angular Interceptor?

- interceptors in Angular are services that implement the `HttpInterceptor` interface.
- They are built for intercepting HTTP client-server communication.
- Interceptors sit between the `HttpClient` interface and the `HttpBackend`.



Chain of interceptor

- Angular Interceptors provide a mechanism to inspect and transform HTTP requests and responses globally before they are sent to the server or processed by the client.
- They act as a middleware layer, allowing developers to centralize common tasks related to HTTP communication.
- Purpose:
 - Interceptors are used to handle cross-cutting concerns related to HTTP requests and responses, such as:
 - Adding common headers (e.g., authorization tokens).
 - Logging requests and responses for debugging.
 - Centralized error handling.
 - Modifying request or response bodies.

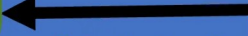
Application
(Client)



Original
Request



Mutated
Response



Chain of Interceptors

Interceptor #1

Interceptor #2

...

Interceptor #N

HttpBackend

Mutated
Request



Original
Response



Backend
(Server)

Implementation

To create an interceptor, a service needs to implement the `HttpInterceptor` interface from `@angular/common/http`.

This interface requires the implementation of an `intercept()` method.

```
import { Injectable } from '@angular/core';

import {
  HttpInterceptor,
  HttpRequest,
  HttpHandler,
  HttpEvent
} from '@angular/common/http';
import { Observable } from 'rxjs';

}
```

```
@Injectable()
export class MyInterceptor implements HttpInterceptor {

  intercept(
    req: HttpRequest<any>,
    next: HttpHandler
  ): Observable<HttpEvent<any>> {
    // Modify the request here, e.g., add a header
    const modifiedReq = req.clone({
      setHeaders: { 'X-Custom-Header': 'Value' }

      // Pass the modified request to the next handler in the
chain
      return next.handle(modifiedReq);
    }
  }
}
```

Angular 17

Angular 17 provides two primary ways to implement HTTP interceptors:

- functional interceptors and class-based (DI-based) interceptors. Both allow you to inspect and transform HTTP requests and responses, but they differ in their structure and how they leverage Angular's Dependency Injection (DI) system.

1. Functional Interceptors:

- Structure:
- Functional interceptors are plain JavaScript functions that implement the `HttpInterceptorFn` interface. They receive the `HttpRequest` and `HttpHandlerFn` as arguments.
- Dependency Injection:
- They can leverage Angular's `inject()` function within their body to obtain dependencies (e.g., services, configuration values). This allows them to access other parts of your application without being a class themselves.
- Registration:
- Functional interceptors are registered within the `provideHttpClient()` function, typically in `app.config.ts` for standalone applications, using `withInterceptors([])`.

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {  
  const authService = inject(AuthService);  
  const authToken = authService.getToken(); // Get token from service  
  if (authToken) {  
    const clonedReq = req.clone({  
      headers: req.headers.set('Authorization', `Bearer ${authToken}`)  
    });  
    return next(clonedReq);  
  }  
  return next(req);  
};
```

Class-Based (DI-Based) Interceptors:

- These are injectable classes that implement the `HttpInterceptor` interface. They have an `intercept` method that receives the `HttpRequest` and `HttpHandler` as arguments.
- Dependency Injection:
 - Dependencies are injected through the class constructor, similar to other Angular services.
- Registration:
 - Class-based interceptors are registered as multi-providers in the providers array of your module or `app.config.ts` (for standalone) using `provideHttpClient(withInterceptorsFromDi())` and `HTTP_INTERCEPTORS`.
 -

// functional implementation

```
export const loggingInterceptor: HttpInterceptorFn = (req, next) => {  
  console.log('Request URL:', req.url);  
  //return next(req);  
  return next(req).pipe(  
    tap(event => {  
      // Response handling  
      // Check for HttpEventType to ensure it's a response  
      if (event.type === 4) { // HttpEventType.Response  
        console.log('Incoming Response:', event);  
        console.log('Incoming Response:', event.body,event.status);  
      }  
    })  
  );  
};
```


tap method

In Angular HTTP Interceptors, the tap operator from RxJS is commonly used to perform side effects on the observable stream of an HTTP request or response without altering the values themselves.

Purpose of tap in Interceptors:

The tap operator allows you to:

Log information:

use tap to log details about the request or response, such as headers, body, or status codes, for debugging or monitoring purposes.

Perform side effects:

It enables you to trigger actions that don't directly modify the data flowing through the observable, like showing a loading spinner before a request and hiding it after the response, or storing data in a service.

Handle errors (without rethrowing):

While catchError is for handling and potentially recovering from errors, tap can be used to perform actions when an error occurs (e.g., logging the error) without preventing the error from propagating further down the stream.

```
export const ErrorInterceptor: HttpInterceptorFn = (req: HttpRequest<unknown>, next: HttpHandlerFn) => {  
  return next(req).pipe(  
    catchError((error: HttpErrorResponse) => {  
      // Handle the error here  
      console.error('error occurred:', error);  
      //throw error as per requirement  
      return throwError(error);  
    })  
  );  
};
```